Important viva question answer from ERP:-

JSP (JavaServer Pages)

1. What is JSP and how does it differ from Servlets?

JSP is a server-side technology that allows embedding Java code in HTML. It differs from Servlets by enabling easier creation of dynamic content using HTML-like syntax.

2. Explain the lifecycle of a JSP page.

The JSP lifecycle includes translation, compilation, initialization, execution, and destruction.

3. What are JSP directives and what are their types?

JSP directives provide instructions to the JSP container and include page, include, and taglib.

4. How do you handle exceptions in JSP?

Exceptions in JSP can be handled using the errorPage and isErrorPage attributes or through custom error pages.

5. What is the difference between <jsp:include> and <jsp:forward>?

<jsp:include> includes content at runtime, while <jsp:forward> forwards the request to another resource.

Java Servlets

6. What is a Servlet and how does it work?

A Servlet is a Java program that handles HTTP requests and responses on a web server, generating dynamic content.

7. Explain the Servlet lifecycle.

The Servlet lifecycle consists of init(), service(), and destroy() methods.

8. What is the difference between doGet() and doPost() methods in a Servlet?

doGet() handles HTTP GET requests (less secure, limited data), while doPost() handles HTTP POST requests (more secure, no data size limits).

9. How does session management work in Servlets?

Session management can be handled using cookies, URL rewriting, HttpSession, or hidden form fields.

10. What is a Servlet Filter and how is it used?

A Servlet Filter is used to intercept requests and responses, often for tasks like logging, authentication, and data compression.

JDBC (Java Database Connectivity)

11. What is JDBC and why is it used?

JDBC is an API that allows Java applications to interact with databases using SQL queries.

12. Explain the different types of JDBC drivers.

The four types are: Type-1 (JDBC-ODBC Bridge), Type-2 (Native API), Type-3 (Network Protocol), and Type-4 (Thin Driver).

13. What are the main steps to connect a Java application to a database using JDBC?

Steps include loading the driver, establishing a connection, creating a statement, executing queries, and closing the connection.

14. What is the purpose of the PreparedStatement and how does it differ from Statement?

PreparedStatement is used for executing precompiled SQL statements, which improves performance and prevents SQL injection compared to Statement.

15. What are transactions in JDBC and how do you manage them?

Transactions ensure data integrity by grouping multiple operations; they are managed using commit() and rollback() methods.

Hibernate

16. What is Hibernate and what are its advantages over JDBC?

Hibernate is an ORM (Object-Relational Mapping) tool that simplifies database operations by mapping Java objects to database tables, eliminating boilerplate code.

17. Explain the concept of Session and SessionFactory in Hibernate.

SessionFactory creates Session instances, which represent connections to the database and are used to perform CRUD operations.

18. What is HQL (Hibernate Query Language) and how is it different from SQL?

HQL is a query language similar to SQL but works with Hibernate objects and properties, not directly with database tables.

19. What are the different types of inheritance mapping in Hibernate?

The types include Table per Class, Table per Subclass, and Table per Concrete Class.

20. How does Hibernate manage caching and what are the types of caching it supports?

Hibernate uses two levels of caching: First-level (default session cache) and Second-level (shared cache). It also supports query caching.

Spring

1. What is the Spring Framework, and what are its key features?

Spring is a Java framework for building enterprise applications with features like dependency injection, AOP, and data access.

2. Explain the concept of dependency injection in Spring.

Dependency injection is a design pattern where the framework manages the dependencies between objects automatically.

3. What are the different types of Spring bean scopes?

Common scopes include singleton, prototype, request, session, and application.

4. What is the difference between @Component, @Service, @Repository, and @Controller annotations in Spring?

They are specialized annotations for different layers: @Component (generic), @Service (business logic), @Repository (data access), @Controller (web controller).

5. How does Spring handle transaction management?

Spring manages transactions using declarative annotations like @Transactional or programmatic approaches.

Spring Boot

6. What is Spring Boot and how does it differ from the traditional Spring Framework?

Spring Boot simplifies application development by providing default configurations and an embedded server.

7. What are Spring Boot starters, and how do they simplify the development process?

Starters are pre-configured dependencies that simplify adding functionalities like web, data access, or security to a Spring Boot project.

8. What is the purpose of the application.properties or application.yml file in a Spring Boot project?

These files are used to configure application settings like database connection details, server port, and other properties.

9. How does Spring Boot's auto-configuration mechanism work?

Auto-configuration automatically sets up beans and configurations based on the dependencies in the project's classpath.

10. What is the role of the embedded server in Spring Boot, and which servers are commonly used?

The embedded server allows applications to run without needing external server setup. Common servers are Tomcat, Jetty, and Undertow.